Data Mining for Big Data - part of B. Jeudy

Each part (this one and C. Largeron part) must be done on a different paper.

It is important to give justifications for all your answers.

1 Map-Reduce (7 pts)

The aim of this exercise is to compute TF-IDF for words in a collection of documents. In this exercice, (a simplified version of) the TF-IDF of a word i in a document j is defined as $\text{tfidf}(i,j) = \text{tf}(i,j) \times \text{idf}(i)$. Term frequency $\text{tf}(i,j)$ is the number of occurrences of word i in document j and the inverse document frequency of i is : $\text{idf}(i) = \log(N/n_i)$ where N is the total number of documents and n_i is the number of documents that contain word i.

Each document in the collection is split into lines. The input RDD named `input` consists of triplets (d, l, str) where d is the document id (a number), l is the line number in document d and str is the string. For instance, we could have a RDD `input` of two documents distributed on two nodes:

Node 1:
```
(1, 3, "apple banana")
(1, 4, "cherry apple banana")
(2, 3, "cherry")
(2, 4, "apple cherry")
(2, 5, "apple")
```

Node 2:
```
(1, 1, "apple banana")
(1, 2, "cherry apple banana")
(1, 3, "banana")
(2, 1, "apple banana cherry")
(2, 2, "apple apple apple")
```

Question 1 : First, we must count how many times each word appear in each document (the $tf(i,j)$). The input of this question is the RDD `input`. Write the code to create a RDD called `tf` containing tuples $((word, d), occ)$ where occ is the number of occurrences of $word$ in document d. You can use the function `split()` to split strings into a list of words. The only Spark functions available for this question are: `map`, `flatMap` and `reduceByKey`.

The RDD on which the `reduceByKey` is applied is called `words`. Your code should construct this RDD.

Question 2 : Write the tuples of the `words` RDD (defined in previous question) computed on Node 1 on the example.

Question 3 : Draw the computation tree(s) of the reduce operation on word `apple`. (On both nodes). How many tupples are send on the network during the shuffle step (just for word `apple`, without any "pre-reduce" optimization).

Question 4 : Same question, but assuming now that a "pre-reduce" optimization happen on each node. Identify in your tree(s) which parts are computed on each node. How many tupples are send on the network during the shuffle step (just for word `apple`).

Question 5 : Write the code to create a RDD called `idf` containing tuples $(word, idf)$ where `idf` is equal to $idf(word)$. You can use the number N of documents in your functions. The only Spark functions available for this question are: `map`, `flatMap` and `reduceByKey`.

Question 6 : Finally, write the code to create a RDD called `tf_idf` containing triplets $(word, d, \text{tfidf}(word, d))$. The only Spark functions available for this question are: `map`, `flatMap`, `reduceByKey` and `join`.

If A and B are two RDDs of pairs (k, b) and (k, c), then `A.join(B)` is a RDD of pairs $(k, (b, c))$ with all pairs (b, c) for each key k.

next exercise on back

2 Locality Sensitive Hashing (3 pts)

In the following matrix, each column represent a set (or document) and each row an element (or shingle). There is a "1" in row i and column j if element i belongs to set j.

	A	B	C	h_1	h_2
0	0	1	1	4	6
1	0	1	0	2	2
2	0	1	1	3	5
3	1	0	1	5	4
4	0	0	0	0	1
5	1	0	0	1	0
6	1	1	1	6	3

Question 7 : Give the formulas and the value of the Jaccard similarities between sets A and B, B and C, and A and C.

Question 8 : We use the hash functions h_1 and h_2 in the table above to define permutations of rows/elements (for instance, row 0 means $h_1(0) = 4$, $h_2(0) = 6$). Using the row by row (optimized) algorithm, compute the minhash of A, B and C according to permutations h_1 and h_2 (give the table of signature after each row).

Recap of Row by row optimized algorithm:

```
For each set S and for each hash function H:
   Initialize signature(S, H) to infinity.
For each row R in increasing order
   For each set S and for each hash function H:
      if S(R)=1 then signature(S, H) = min(signature(S, H), H(R))
```

S(R) denotes the value of set S in row R. At the end of the algorithm, signature(S, H) is the minhash value of set S for function H.